Generic Urban Data Integration Platform
Group 11

1 Study the Data

Dataset	Primary entity	Primary key	Likely join attributes
Taxi trips	One taxi trip	Generated `trip_id`	`tpep_pickup_datetime`, `tpep_dropoff_datetime`, `PULocationID`, `DOLocationID`
Taxi zones	One NYC taxi zone	`LocationID`	`LocationID` (as a foreign key in other tables)
Weather	One hourly weather observation	(`year`, `month`, `day`, `hour`)	(`year`, `month`, `day`, `hour`)
Air quality	Hourly air quality measurement	Generated `observation_id`	(`Date GMT`, `Time GMT`)

Since the taxi trips and air quality datasets didn't contain any primary keys, nor could any be created from existing columns, a decision was made to generate new ones. More specifically, the taxi trips dataset, even though extremely unlikely, could theoretically contain multiple rows with the exact same values, and thus none of the attributes could confidently be used as unique identifiers. The same is true for the air quality observations.

1.1 Temporal, Categorical, and Growing attributes

- **Taxi trips**
  - Temporal: `tpep_pickup_datetime`, `tpep_dropoff_datetime`.
  - Categorical: `VendorID`, `RatecodeID`, `store_and_fwd_flag`, and `payment_type`.
  - Likely to grow: `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `passenger_count`, `trip_distance`, `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount`, `congestion_surcharge`, and `airport_fee`.
- **Taxi zones**
  - Temporal: None.
  - Categorical: `Borough` and `service_zone` (even though the Zone attribute has a finite set of values that are pre-defined, since every location is mapped to a specific Zone with an (almost) 1-1 relationship, we will treat it as a non-categorical value).
  - Likely to grow: None.
- **Weather**
  - Temporal: `year`, `month`, `day`, and `hour`.
  - Categorical: `temp_source`, `rhum_source`, `prcp_source`, `snwd_source`, `wdir_source`, `wspd_source`, `wpgt_source`, `pres_source`, `cldc_source`, `coco_source`, `cldc`, and `coco`.
  - Likely to grow: `year`, `temp`, `prcp`, `snwd`, `wspd`, `wpgt`, and `pres`.
- **Air quality**
  - Temporal: `Date Local`, `Time Local`, `Date GMT`, `Time GMT`, and `Date of Last Change`.
  - Categorical: `State Code`, `County Code`, `Site Num`, `Parameter Code`, `POC`, `Parameter Name`, `Units of Measure`, `Qualifier`, `Method Code`, `Method Type`, `State Name`, `County Name`, and `Datum`.
  - Likely to grow: `Date Local`, `Time Local`, `Date GMT`, `Time GMT`, `Latitude`, `Longitude`, `Datum`, `Date of Last Change`, `Uncertainty`, `Sample Measurement`, `MDL`.

2 Design Your Storage Architecture

2.1 Design

2.1.1 Directory structure

The local repository separates source files, configurations, and reusable Spark jobs:

```
Data-Ingestion-Framework/
|-- configs/
|   `-- *.json
|-- landing/
|   |-- taxi/
|   |-- taxi_zone_lookup/
|   |-- weather/
|   `-- air_quality/
`-- jobs/
    `-- *.py
```

The `configs/` directory contains dataset-specific rules. The `data/` directory acts as the landing area for source files, while `jobs/` contains reusable Spark pipeline components.

2.1.2 Delta table organization

Each dataset is stored independently as a Delta table. We use a *Medallion Architecture*, and thus we have created three layers: **Bronze** layer preserves the source data and lineage metadata, **Silver** stores standardized and validated records, and **Gold** stores the integrated analytical dataset.

Layer	Table	Purpose
Bronze	bronze_taxi_trips	Raw taxi data with metadata
Bronze	bronze_weather	Raw weather data with metadata
Bronze	bronze_air_quality	Raw air-quality data with metadata
Bronze	bronze_taxi_zones	Raw zone lookup with metadata
Silver	silver_taxi_trips	Standardized and validated taxi data
Silver	silver_weather	Standardized and validated weather data
Silver	silver_air_quality	Standardized and validated air-quality data
Silver	silver_taxi_zones	Standardized and validated zone lookup
Metadata	ingestion_runs	Audit record for each ingestion run
Gold	integrated_taxi_trips	Integrated/Aggregated dataset for analysis

Table 1: Delta table organization.

2.1.3 Naming conventions

The Bronze layer preserves original source column names, while Silver and Gold use lowercase `snake_case` names. Dataset Delta tables follow the `<layer>_<dataset>` convention, for example `silver_taxi_trips`. Meaningful names are used when source names are unclear, such as `PULocationID` becoming `pickup_zone_id`. Identifiers, timestamps, and dates use the suffixes `_id`, `_datetime`, and `_date`, respectively.

2.1.4 Partitioning strategy

The Delta tables use dataset-specific partitioning strategies:

- **Taxi trips:** partitioned by `pickup_date`.
- **Weather:** not partitioned because the hourly dataset is relatively small.
- **Air quality:** partitioned by `local_date`.
- **Taxi zones:** not partitioned because it is a small lookup table.

It should also be noted that Delta Lake best practices recommend partitioning by a column if you expect the partition to be at least 1 GB. Even though the given data are not large enough to achieve this, the pipeline enforces the above partitions for assignment purposes.

2.1.5 Design decisions

Which datasets should conceptually be treated as lookup tables? The taxi-zone lookup is conceptually a lookup table because it maps each `location_id` to a borough and zone.

Which datasets should not be partitioned? Explain why. Taxi zones and weather are not partitioned because they are small datasets.

Which datasets require different partitioning strategies? Both Taxi trips and Air-quality are partitioned by date attributes (`pickup_date`, `local_date`) to support date-range queries. A different, more efficient, partitioning strategy for the gold layer is discussed in the benchmark report.

Under what conditions does partitioning become harmful? Partitioning is harmful when it creates many small files, produces uneven partitions, or uses columns that queries rarely filter on. High-cardinality columns such as trip identifiers and exact timestamps should not be used as partition columns.

If the total data volume increased by 20×, what changes would you make to your storage design? The Bronze-Silver-Gold structure would remain unchanged. The partitioning scheme should be reconsidered for air-quality and taxi-trips datasets, based on the new data. Taxi zones and weather would remain unpartitioned.

2.2 Implementation of the storage architecture

We designed our solution to be self-contained and easy to run. For this reason, the whole architecture runs inside a Docker container. The container includes an Apache Spark cluster consisting of one master node and two worker nodes. In addition, MinIO is used as an object (blob) storage system, providing the storage layer required by Delta Lake for writing and managing Delta tables. Finally, a Jupyter server is included to speed up the development, experimentation, and testing process (setting up the Jupyter connection is described in `notebooks/Readme.md`).

3 Build a Generic Ingestion Framework

3.1 Configurable ingestion framework

The framework uses one Spark pipeline to ingest CSV and Parquet sources (but can be used with other file types as well), validate raw schemas, apply configured quality checks and transformations, write Delta tables, and record ingestion metadata.

1. **Generic and reusable components.** The reusable components are the Spark-session setup, configuration loading, file reading, schema validation, operation dispatcher, quality-check runner, Delta writers, and metadata writer. These components are shared by all four datasets. Even though an ingest module creates a single complete pipeline (bronze, silver & gold writing), each job can also be invoked independently.

2. **Dataset-specific components.** Each dataset has its own JSON configuration containing source paths and formats, expected schema, rename mappings, timestamp rules, quality rules, and schema version. These rules remain dataset-specific because the source schemas and business meanings differ.

3. **Transformation-rule management.** Transformation rules are stored as ordered operations in JSON files. For example, taxi trips rename source fields and convert New York timestamps to UTC, while weather creates an hourly timestamp. Quality checks like null and duplicate record handling operations are also dataset-specific (since different dataset might want to handle them differently) and follow a similar structure. The generic operation and quality dispatcher applies these rules without dataset-specific ingestion scripts. One of these generic quality rules is **validate_duplicates**, which checks for duplicate records based on a list of columns. All the quality rules also contain an *action* field that specifies what should happen if any discrepancy is found (e.g., *fail*, *drop*, etc.).

4. **Metadata management.** Metadata is stored at two levels. First, every Bronze record receives `source_file_path`, `ingested_at`, and `ingestion_run_id`. These fields show which source file produced a record, when it was ingested, and which pipeline execution processed it.

   Second, the `metadata` Delta table stores one record per successful ingestion run. Its fields are `run_id`, `dataset`, `schema_version`, `processed_rows`, `rejected_rows`, `started_at`, `finished_at`, `duration_seconds`, and `status`. This provides an audit trail for monitoring pipeline executions and comparing runs over time.

5. **Reduced duplication and maintenance.** Common ingestion logic is implemented once rather than copied into four scripts. Changes to validation, transformation, and quality-checking logic are limited to changing or adding a JSON configuration file.

6. **Adding 20 datasets.** Most new datasets would require a new JSON configuration and no changes to the generic pipeline. New Python code would be needed only when a dataset requires a previously unsupported transformation or quality-checking rule.

4 Design a Common Data Model

The common model is applied in the Silver and Gold layers so that the datasets can be combined consistently.

- **Timestamps:** All derived timestamps use UTC.
- **Names:** Silver and Gold use lowercase `snake_case`. Identifiers, timestamps, and dates use `_id`, `_datetime`, and `_date`.
- **Missing values:** Bronze preserves missing source values for traceability. In Silver, missing primary-key values cause the pipeline to fail, while other missing values are retained and reported as warnings. The framework also supports dropping or filling missing values when configured.
- **Types:** Numeric identifiers and counts use integral types, while source codes that may contain leading zeros remain `string`. Continuous measurements and coordinates use `double` where required. Dates use `date`, and the common event-time fields use `timestamp`.

Dataset	Transformations
Taxi trips	Rename source columns to obey naming conventions, generate `trip_id`, convert `pickup_datetime` and `dropoff_datetime` to UTC, and create date columns with year-month-date for partitioning.
Taxi zones	Rename `LocationID`, `Borough`, and `Zone` to `location_id`, `borough`, and `zone`.
Weather	Drop `snwd`, `snwd_source`, `wpgt`, and `wpgt_source`, which are not used in the common model; create `observation_datetime`; standardize column names.
Air quality	Drop `Uncertainty`; rename date/time fields; standardize names; generate `observation_id`; create `observation_datetime`; cast coordinates, measurements, and dates.

Table 2: Transformations used to create the common data model.

The common decision that was made regarding the handling of invalid data across datasets is the following: Duplicates and records that have attribute values outside the allowed range (e.g., negative values in columns that are not allowed) are dropped, while any invalid primary key (null or duplicate) fails the ingestion. Null values in any columns are currently allowed, but a null-checking rule has been added on important columns, in order for the user to be warned about their existence.

5 Build the Integration Pipeline

The pipeline creates the Delta table `gold_integrated_taxi_trips`. Each row represents one taxi trip enriched with pickup and drop-off information, weather conditions, and air-quality measurements.

1. **Weather association.** Pickup and drop-off timestamps are truncated to the hour (`pickup_datetime`, `dropoff_datetime`) and left-joined with the corresponding hourly weather observations (`observation_datetime`). Weather attributes are stored separately with `_pu` and `_do` suffixes.
2. **Air-quality association.** Air-quality records are filtered to New York and aggregated to one record per `observation_datetime`. The reason behind that is that the granularity of the data in the `taxi_zones` is by zone, but the data in `air_quality` is by longitude and latitude for a specific date and there is no mapping provided between the two (also there are trips where the corresponding zone from the lookup is unknown, meaning for those we cannot get precise weather data). The pipeline calculates the average measurement for a specific hour (`observation_datetime`) and joins it to the pickup and dropoff hour (`pickup_datetime`, `dropoff_datetime`).
3. **Missing observations.** All integration joins are left joins. Therefore, every taxi trip remains in the Gold table even when zone, weather, or air-quality context is unavailable. Missing contextual fields are stored as `NULL`.
4. **Limitations.** The hourly join cannot represent conditions that change within an hour. Air quality is averaged across New York records for each hour, so it does not represent the exact taxi pickup or drop-off zone. The available datasets also do not provide a direct mapping between taxi zones and air-quality monitoring locations. If an external service (e.g., an API) or extra data that can provide us with such a mapping existed, the results could have been more accurate.